

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

OUTPUT :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char cmd[100];

    printf("Enter Linux command: ");
    scanf("%s", cmd);
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child PID : %d\n", getpid());
        execlp(cmd, cmd, NULL);
        perror("Execution Failed");
    }
    else if(pid > 0)
    {
        printf("Parent PID : %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }
    else
    {
        printf("Fork Failed\n");
    }
    return 0;
}
```

```
hemaswi@Hemaswi: ~  
hemaswi@Hemaswi:~$ nano practical1.c  
hemaswi@Hemaswi:~$ gcc practical1.c -o practical1  
hemaswi@Hemaswi:~$ ./practical1  
Enter Linux command: ls  
Parent PID : 1178  
Child PID : 1179  
Demandings          multiplechildprocesses.c  
Demandings.c        output.txt  
Ex1                  parentchild  
OSLab                parentchild.c  
OSSP_Project         parentchildexecutingconcurrently  
OSLab                parentchildexecutingconcurrently.c  
Virtualaddress       parentchildpid  
Virtualaddress.c    parentchildpid.c  
a                    parentwaitchild  
a.txt                parentwaitchild.c  
b                    physical_address  
b.txt                physical_address.c  
childparentprocess  practical1  
childparentprocess.c practical1.c  
displayprocess      practical2  
displayprocess.c    practical2.c  
example.txt         practical7  
executelinux        practical7.c  
executelinux.c      practical12  
fill1.c             practical12.c  
file.txt            priority_scheduler  
file1               process_making  
file1.c             process_making.c  
file1.txt           processid  
file2               processid.c  
fork                readuser  
fork.c              readuser.c  
forkexec            s2commands  
forkexec.c          script  
hello               script.sh  
hello.c             sh  
input.txt           shortcut  
main.c              virtualadress.c.save  
malloc              wait  
malloc.c            wait.c  
memory_accessing    waiting  
memory_accessing.c  waiting.c  
multiplechildprocesses  
Child process completed.
```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

OUTPUT:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int src, dest;
    char buffer[1024];
    int bytes;

    src = open("input.txt", O_RDONLY);

    if(src < 0)
    {
        printf("Cannot open source file\n");
        return 1;
    }

    dest = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    while((bytes = read(src, buffer, sizeof(buffer))) > 0)
    {
        write(dest, buffer, bytes);
    }

    close(src);
    close(dest);

    printf("File copied successfully.\n");

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano practical2.c
hemaswi@Hemaswi:~$ gcc practical2.c -o practical2
hemaswi@Hemaswi:~$ ./practical2
File copied successfully.
hemaswi@Hemaswi:~$
```